

INFORME DEL PROYECTO:

Predicción y Análisis de la Calidad del Vino Tinto



Asignatura: Programación para la Ciencia de Datos
(SCY1101)

Evaluación: Evaluación Parcial 2
Integrante: Belén Toloza Concha
Fecha: 19/05/2026

1. Resumen Ejecutivo de mi trabajo.....	3
2. Conociendo los datos: El Dataset y sus Variables.....	3
3. Metodología: ¿Cómo trabajé?.....	4
4. Fase 1: Limpiando y Explorando los Datos.....	4
El problema de los duplicados.....	5
Analizando los Outliers (Valores extraños).....	5
El Desbalanceo de las notas.....	7
5. Fase 2: Preparando el terreno para la IA.....	7
Escalamiento de datos.....	7
Análisis de los resultados del Árbol de Decisión.....	8
6. Fase 3: Evaluando mis Modelos Supervisados.....	9
7. Fase 4: Mejorando a mi modelo ganador.....	10
Los ajustes que elegí.....	10
La trampa de la Exactitud.....	11
¿Qué aprendí y qué podemos hacer a futuro?.....	11
8. Fase 5: El experimento a ciegas (Aprendizaje No Supervisado).....	11
Buscando los grupos y haciendo un "Re-framing".....	12
El Mapa Final con PCA (Reduciendo las dimensiones).....	12
Mi gran descubrimiento: El Vino Premium vs Económico.....	13
Mi conclusión final de esta fase.....	13
9. Conclusiones y Lecciones Aprendidas.....	13
Dificultades y Recomendaciones a futuro.....	14

1. Resumen Ejecutivo de mi trabajo

El objetivo de este proyecto fue analizar un conjunto de datos (dataset) llamado ["Red Wine Quality"](#) de Kaggle, que contiene información sobre las características químicas de miles de botellas de vino tinto de la variante "Vinho Verde" de Portugal. Mi meta principal fue construir un modelo de Inteligencia Artificial que pudiera predecir **qué nota de calidad tiene un vino basándose únicamente en sus químicos** (como el alcohol, el azúcar o la acidez), sin que un humano tenga que probarlo.

A lo largo del proyecto pasé por varias etapas. Primero **limpié los datos** porque venían con filas repetidas y valores atípicos. Luego, **entrené dos modelos de Machine Learning** (un Árbol de Decisión y un Random Forest) para ver cuál era mejor adivinando la calidad. Me di cuenta de que el Random Forest era mucho más **seguro** y no se confundía tanto, así que lo elegí como ganador y lo mejoré aún más.

Finalmente, hice un experimento sin darle las notas de calidad a la máquina, usando **Aprendizaje No Supervisado**. Fue un éxito total, ya que el algoritmo logró agrupar a los vinos por sí solo y descubrió exactamente cuál es el perfil químico de los vinos de categoría **"Premium"**. A continuación, explicaré el paso a paso de todo mi proceso.

2. Conociendo los datos: El Dataset y sus Variables

Antes de programar, era fundamental entender qué estaba analizando. El dataset cuenta con **1.599** ejemplos de vinos (filas). Por cada botella, se tienen **12 columnas en total**: 11 son mediciones fisicoquímicas y 1 es la nota final.

Para que mi modelo predictivo tuviera sentido, analicé qué significaba cada uno de estos químicos:

1. **Acidez Fija (Fixed Acidity):** Son los ácidos principales que le dan la estabilidad al sabor del vino.
2. **Acidez Volátil (Volatile Acidity):** Es la cantidad de ácido acético. Esto es clave porque si está muy alto, el vino toma un sabor desagradable a vinagre.
3. **Ácido Cítrico (Citric Acid):** Le aporta frescura al vino.
4. **Azúcar Residual (Residual Sugar):** Es el azúcar que sobra después de la fermentación. Básicamente dicta qué tan dulce es la botella.
5. **Cloruros (Chlorides):** Es la cantidad de sal en el vino.
6. **Dióxido de Azufre Libre y Total (Free / Total Sulfur Dioxide):** Son los conservantes químicos que se le echan al vino para evitar que se oxide o le crezcan microbios.

7. **Densidad (Density):** Se relaciona con la concentración de componentes del vino respecto al agua.
8. **pH:** Mide qué tan ácido o alcalino es el vino (usualmente va entre 3 y 4).
9. **Sulfatos (Sulphates):** Actúan como antioxidantes.
10. **Alcohol:** El porcentaje de volumen de alcohol en la botella.

Mi Variable Objetivo (Target): La columna número 12 es la **Calidad** (Quality). Esta nota no la inventó una máquina, sino que fue asignada por al menos tres catadores expertos humanos mediante pruebas a ciegas, usando una escala del 0 (muy malo) al 10 (excelente). Mi desafío en este trabajo fue lograr que el modelo de Python **aprenda a deducir esta nota** usando únicamente las 11 variables químicas anteriores.

3. Metodología: ¿Cómo trabajé?

Para hacer todo esto, como hemos visto en clases, utilicé el lenguaje de programación Python y notebooks .ipynb en Google Colab. Usé las distintas librerías que hemos trabajado que son súper útiles para la Ciencia de Datos:

- **Pandas:** La usé para ver las tablas, filtrar los datos y limpiar lo que estuviera malo.
- **Seaborn y Matplotlib:** Las usé para dibujar todos los gráficos y entender los datos de forma visual.
- **Scikit-Learn:** De aquí saqué toda la matemática y los algoritmos para que la máquina aprendiera a predecir.

Mi trabajo se dividió en dos grandes mundos. Primero, usé el **Aprendizaje Supervisado**, que, como hemos visto en clases, es cuando se le enseña a la máquina pasándole los datos químicos y diciéndole "este vino tiene nota 5" o "este tiene nota 8" para que aprenda las reglas. Después, usé el **Aprendizaje No Supervisado**, que es cuando le paso los datos ciegos (sin la nota) y dejo que la máquina busque patrones por su cuenta. Para esto, a la máquina le pasé el 70% de los datos para poder usar el Aprendizaje Supervisado y oculté 30% de los datos para posteriormente probar que la máquina adivine bien con Aprendizaje no Supervisado.

4. Fase 1: Limpiando y Explorando los Datos

Lo primero que hice al cargar mi archivo fue revisarlo para ver si venía en buenas condiciones. Usé comandos básicos como .info() y .describe() para hacer un escaneo rápido de las columnas.

```

1 df.info()
2 df.isnull().sum()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   fixed acidity          1599 non-null   float64
1   volatile acidity       1599 non-null   float64
2   citric acid            1599 non-null   float64
3   residual sugar         1599 non-null   float64
4   chlorides              1599 non-null   float64
5   free sulfur dioxide    1599 non-null   float64
6   total sulfur dioxide   1599 non-null   float64
7   density               1599 non-null   float64
8   pH                    1599 non-null   float64
9   sulphates             1599 non-null   float64
10  alcohol               1599 non-null   float64
11  quality               1599 non-null   int64   
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

```

```

1 print(df.describe())

```

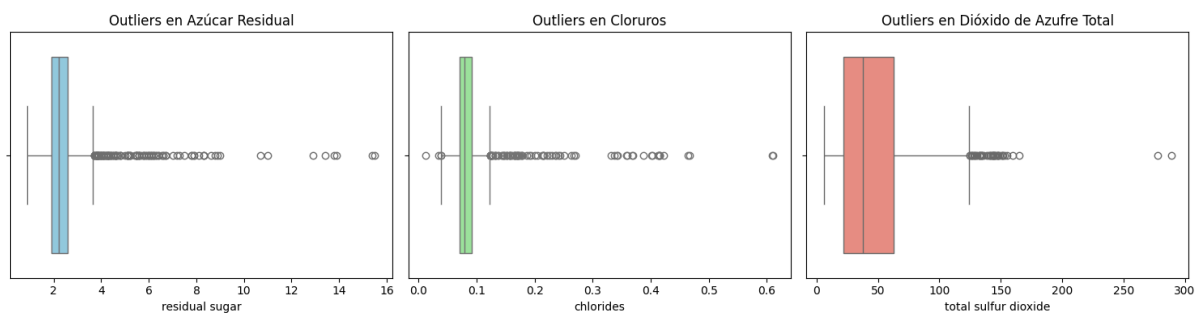
	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000	1359.000000
mean	8.310596	0.529478	0.272333	2.523400	0.088124	15.893304	46.825975	0.996709	3.309787	0.658705	10.432315	5.623252
std	1.736990	0.183031	0.195537	1.352314	0.049377	10.447270	33.408946	0.001869	0.155036	0.170667	1.082065	0.823578
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	8.400000	3.000000
25%	7.100000	0.350000	0.090000	1.900000	0.070000	7.000000	22.000000	0.995600	3.210000	0.550000	9.500000	5.000000
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000	0.996700	3.310000	0.620000	10.200000	6.000000
75%	9.200000	0.640000	0.430000	2.600000	0.091000	21.000000	63.000000	0.997820	3.400000	0.730000	11.100000	6.000000
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289.000000	1.003690	4.010000	2.000000	14.900000	8.000000

El problema de los duplicados

Al revisar mi tabla, me di cuenta de que **no habían valores nulos** o vacíos, lo cual fue un alivio. Sin embargo, al usar una función para buscar repetidos, encontré que habían aproximadamente **240 filas que estaban duplicadas**. Decidí eliminar estos duplicados de inmediato. Si los dejaba, el modelo iba a estudiar la misma información dos veces y se iba a memorizar los datos, engañándome con resultados que parecerían perfectos pero que en la realidad estarían **sesgados**.

Analizando los Outliers (Valores extraños)

Después revisé si habían "Outliers", que son valores muy alejados de lo normal. Para esto, dibujé Diagramas de Caja (Boxplots) y usé la matemática del Rango Inter cuartílico (IQR) para calcular los límites de la normalidad.



Al ver los gráficos, noté que habían muchos puntos sueltos. Para no quedarme solo con la imagen visual, armé una tabla contando matemáticamente cuántos valores extraños exactos tenía cada componente químico.

	Variable	Total_Outliers	Porcentaje
3	residual sugar	126	9.27
4	chlorides	87	6.40
9	sulphates	55	4.05
6	total sulfur dioxide	45	3.31
0	fixed acidity	41	3.02
7	density	35	2.58
8	pH	28	2.06
5	free sulfur dioxide	26	1.91
1	volatile acidity	19	1.40
10	alcohol	12	0.88
2	citric acid	1	0.07

Al analizar estos números, descubrí dos cosas súper interesantes:

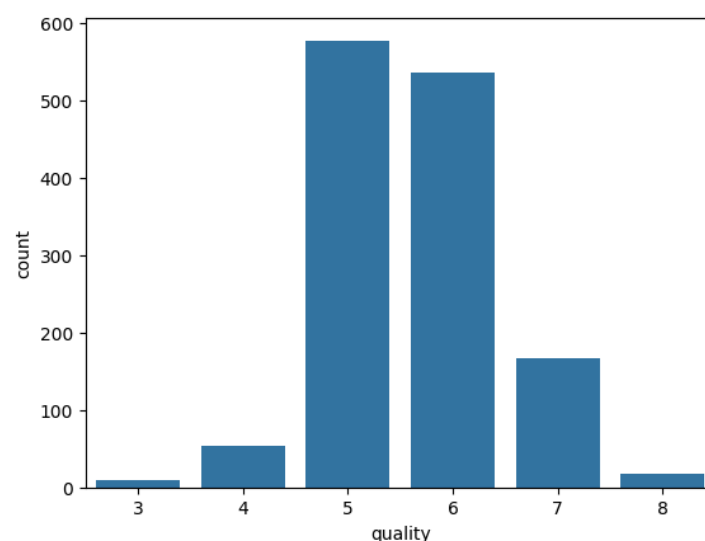
1. Los que ganan en cantidad: El "Azúcar Residual" (residual sugar) y los "Cloruros" (chlorides) son las columnas con mayor frecuencia de outliers, con 126 y 87 casos respectivamente. Esto representa casi el 9.2% y 6.4% de todos los datos.
- 2.
3. Los que ganan en tamaño extremo: Por otro lado, columnas como el "Dióxido de Azufre Total" (total sulfur dioxide) tienen menos cantidad de outliers (solo 45 casos, un 3.3%), pero sus valores máximos son gigantes, saltando bruscamente hasta 289.00.
- 4.

Cualquier persona pensaría en borrarlos de inmediato porque a simple vista parecen errores de tipeo o fallos del sistema. Sin embargo, investigué si estos valores atípicos podían pasar en la vida real. Descubrí que no son errores; existen vinos que legítimamente son mucho más dulces (cosechas tardías), más salados o que requieren mayores conservantes para no oxidarse.

Borrarlos a ciegas sería un gran error, ya que destruiría casi el 10% de la información súper valiosa de los vinos con características únicas. Por lo tanto, tomé la decisión de conservarlos todos en esta etapa, sabiendo que en la Fase 2 usaré herramientas como StandardScaler para escalar estos números y evitar que engañen al modelo.

El Desbalanceo de las notas

Al graficar las notas de calidad, vi un problema gigante: casi el 82% de todos los vinos tenían nota 5 o 6. Los vinos muy malos (nota 3) o muy excelentes (nota 8) casi no existían en la tabla. Esto es súper peligroso, porque un modelo flojo podría simplemente adivinar "nota 5" todo el rato y parecería que es un buen modelo, aunque en realidad no **esté aprendiendo nada**.



5. Fase 2: Preparando el terreno para la IA

Antes de entrenar a la máquina, tuve que hacer un par de ajustes obligatorios. Separé los datos en dos partes: el **70%** de las filas las usé para que el modelo **entrene y estudie**, y el **30%** restante lo escondí para hacerle una prueba final con **datos que nunca antes haya visto**.

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
2 print("Datos para entrenar:", len(X_train))
3 print("Datos para evaluar:", len(X_test))
```

```
Datos para entrenar: 951
Datos para evaluar: 408
```

Escalamiento de datos

Tuve que aplicar la herramienta vista en clases **StandardScaler**. Esto lo hice porque me di cuenta de que algunos químicos (como el dióxido de azufre) tenían valores altísimos, de hasta 300, mientras que otras cosas como los cloruros tenían valores de 0.05. Si le pasaba

los números así a la máquina, iba a pensar que el **dióxido de azufre era más importante sólo porque su número era más grande**. El escalador aplastó todos los números a un rango parejo para que todo fuera justo.

Para automatizar esto y no cometer errores manuales, metí todo en un Pipeline o tubería donde los datos entran crudos y salen transformados directamente hacia el modelo.

```
1 # Construir pipeline para el modelo 1
2 pipe_tree = Pipeline([
3     ('escalador', StandardScaler()),
4     ('modelo', DecisionTreeClassifier(random_state=42))
5 ])
6
7 # Entrenar
8 pipe_tree.fit(X_train, y_train)
9
10 # Predecir y evaluar
11 pred_tree = pipe_tree.predict(X_test)
12 print("--- RESULTADOS ÁRBOL DE DECISIÓN ---")
13 print("Exactitud (Accuracy):", accuracy_score(y_test, pred_tree))
14 print("\nReporte de métricas:\n", classification_report(y_test, pred_tree, zero_division=0))
```

--- RESULTADOS ÁRBOL DE DECISIÓN ---
Exactitud (Accuracy): 0.5098039215686274

Reporte de métricas:

	precision	recall	f1-score	support
3	0.20	0.20	0.20	5
4	0.12	0.15	0.13	13
5	0.65	0.60	0.62	172
6	0.49	0.49	0.49	164
7	0.39	0.44	0.42	50
8	0.00	0.00	0.00	4
accuracy			0.51	408
macro avg	0.31	0.31	0.31	408
weighted avg	0.53	0.51	0.52	408

Análisis de los resultados del Árbol de Decisión

Al revisar las métricas de este primer modelo, me di cuenta de varios problemas graves. Primero, la Exactitud (**Accuracy**) fue de apenas un **51%**. Esto significa que el modelo se está equivocando prácticamente la mitad de las veces al intentar predecir la nota de un vino, lo cual no es útil para nuestro objetivo.

Sin embargo, el verdadero problema se ve al analizar el "Reporte de métricas", específicamente en la columna del F1-Score. Aquí se comprobó mi miedo sobre el desbalanceo de los datos que detecté en la Fase 1:

- El modelo solo funciona de manera decente adivinando los vinos de calidad promedio 5 y 6 (obteniendo un F1-Score de 0.62 y 0.49). Esto pasa porque se aprovechó de que son los vinos que más abundan en la tabla.
- Pero le va pésimo con los vinos menos frecuentes. En los vinos de mala calidad (nota 3 y 4) los puntajes bajan drásticamente a 0.20 y 0.13. Lo más grave ocurre con los vinos de nota 8, donde el F1-Score es un rotundo 0.00. Por lo tanto, el algoritmo no fue capaz de identificar ni un solo vino excelente de manera correcta.

¿Por qué pasó esto? El Árbol de Decisión es un algoritmo muy simple que tiende a sufrir de "Sobreajuste" (Overfitting). Esto quiere decir que tiene una alta varianza; el modelo se

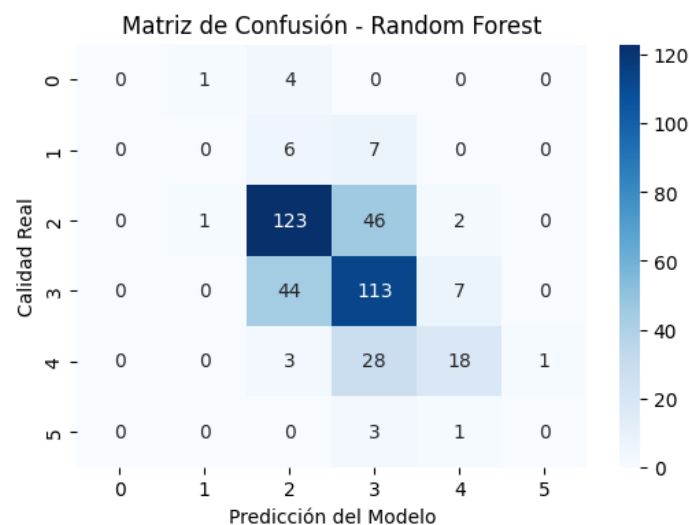
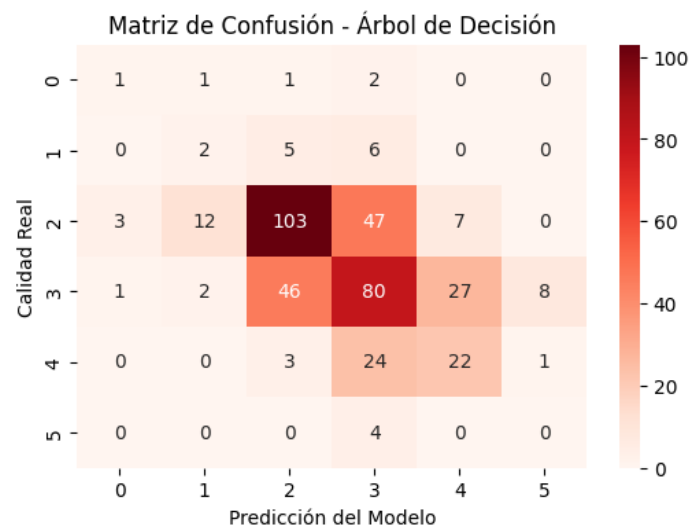
memorizó los vinos promedios de la tabla, pero como no supo generalizar bien las reglas químicas, colapsó por completo al intentar predecir datos o vinos más raros.

Debido a este mal resultado, tomé la decisión de descartarlo como modelo definitivo y probar un algoritmo de ensamble mucho más robusto (como el Random Forest) que pueda manejar mejor la varianza y el desbalanceo de las notas extremas.

6. Fase 3: Evaluando mis Modelos Supervisados

Entrené dos modelos distintos para hacerlos competir entre sí: un **Árbol de Decisión** simple y un **Random Forest** (que es un bosque con muchos árboles votando en conjunto).

Para evaluarlos, no miré solamente su exactitud general (Accuracy), porque ya sabía que el desbalanceo de las notas 5 y 6 me iba a engañar. En su lugar, les apliqué **Validación Cruzada** (hacerles 5 pruebas distintas a cada uno) y miré con lupa la Matriz de Confusión.



Al mirar las matrices, me fijé en los números que estaban fuera de la línea diagonal (que son los errores).

- **El Árbol de Decisión:** Se equivocaba un montón. Por ejemplo, muchos vinos que en realidad eran de calidad 7, el modelo decía que eran de calidad 6. Esto pasa porque el árbol sufre de mucho sobreajuste; memorizó los datos de entrenamiento pero al darle datos nuevos, colapsó.
- **El Random Forest:** Tuvo un desempeño mucho mejor. Al ser un grupo de múltiples árboles trabajando juntos, logró generalizar las reglas químicas. Tuvo una diagonal principal mucho más fuerte y su F1-Score (que es la métrica que usé para que no me engañe el desbalanceo) fue bastante superior.

```
1 print("===== REPORTE: ÁRBOL DE DECISIÓN =====")
2 print(classification_report(y_test, pred_tree, zero_division=0))
3
4 print("\n===== REPORTE: RANDOM FOREST =====")
5 print(classification_report(y_test, pred_forest, zero_division=0))
```

===== REPORTE: ÁRBOL DE DECISIÓN =====				
	precision	recall	f1-score	support
3	0.20	0.20	0.20	5
4	0.12	0.15	0.13	13
5	0.65	0.60	0.62	172
6	0.49	0.49	0.49	164
7	0.39	0.44	0.42	50
8	0.00	0.00	0.00	4
accuracy			0.51	408
macro avg	0.31	0.31	0.31	408
weighted avg	0.53	0.51	0.52	408

===== REPORTE: RANDOM FOREST =====				
	precision	recall	f1-score	support
3	0.00	0.00	0.00	5
4	0.00	0.00	0.00	13
5	0.68	0.72	0.70	172
6	0.57	0.69	0.63	164
7	0.64	0.36	0.46	50
8	0.00	0.00	0.00	4
accuracy			0.62	408
macro avg	0.32	0.29	0.30	408
weighted avg	0.60	0.62	0.60	408

Declaré al Random Forest como el modelo ganador absoluto.

7. Fase 4: Mejorando a mi modelo ganador

Aunque mi modelo Random Forest ya era el mejor, quise sacarle un poco más de rendimiento. Para no estar adivinando números a mano, usé **GridSearchCV**, que es una herramienta de Python que prueba automáticamente muchas configuraciones hasta encontrar la mejor.

Los ajustes que elegí

Armé este "diccionario" de opciones para que el computador las probara:

```

1
2 param_grid = {
3     'modelo__n_estimators': [50, 100, 150],
4     'modelo__max_depth': [None, 10, 15],
5     'modelo__min_samples_split': [2, 5, 10],
6     'modelo__class_weight': ['balanced', 'balanced_subsample'] # Ayuda activamente con el desbalanceo detectado
7 }

```

Elegí esos valores por dos grandes razones:

1. **Para que el modelo no sea tan básico (evitar el subajuste):** Le puse que probara con 50, 100 y 150 árboles. Si le ponía muy poquitos árboles al bosque, el modelo no iba a tener la capacidad de aprender bien los patrones y se iba a equivocar mucho.
2. **Para nivelar la cancha:** Como la inmensa mayoría de los vinos son nota 5 o 6, le agregué la opción `class_weight='balanced'`. Esto obliga al modelo a darle más importancia a los vinos raros y a castigarse más si se equivoca con ellos.

La trampa de la Exactitud

Acá hice un cambio súper importante: le dije a la herramienta que no evaluara a los modelos usando la Exactitud normal (Accuracy). Como mis datos están tan desbalanceados, la exactitud me iba a mentir y a inflar el puntaje.

En su lugar, usé una métrica llamada `scoring='f1_weighted'`. Esto obliga al modelo a buscar un equilibrio de verdad y a no hacerse el tonto ignorando a los vinos que son de calidades extremas.

¿Qué aprendí y qué podemos hacer a futuro?

Después de correr el código, logré un **Accuracy final de 62.5%** (mejor que el 62.25% que tenía al principio con el modelo base). Quedó súper bien configurado para adivinar los vinos intermedios.

El gran desafío: A pesar de las mejoras, las notas muy raras (3, 4 y 8) siguieron marcando un puntaje de 0.00. Al revisar bien, me di cuenta de que esto pasa porque simplemente casi no hay ejemplos de esos vinos en el dataset original. Si le pasamos tan poquitos datos, es imposible que el modelo aprenda a reconocerlos.

Mi propuesta (Re-framing): Para solucionar esta falta de datos en el futuro, propongo dejar de adivinar 6 notas diferentes. Sería mucho mejor agrupar a los vinos en solo 3 grandes categorías: Malo (notas 3 y 4), Regular (notas 5 y 6) y Excelente (notas 7 y 8). Al juntarlos así, los grupos tendrían muchas más muestras y la **Inteligencia Artificial funcionaría muchísimo mejor**.

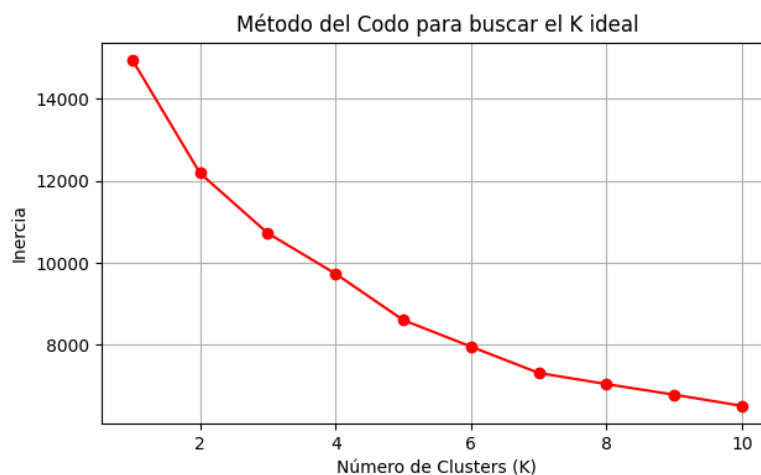
8. Fase 5: El experimento a ciegas (Aprendizaje No Supervisado)

Para ir un paso más allá, quise hacer un experimento donde le oculté por completo las notas de "calidad" a la máquina. La dejé "a ciegas" para ver si el algoritmo lograba armar familias de vinos basándose solamente en si sus químicos se parecían.

Buscando los grupos y haciendo un "Re-framing"

Primero tenía que saber en cuántos grupos dividir los vinos. Inicialmente pensé en buscar muchas categorías de calidad (del 3 al 8), pero como ya sabía que casi no hay vinos con notas extremas, separarlos en tantos grupos iba a salir mal estadísticamente.

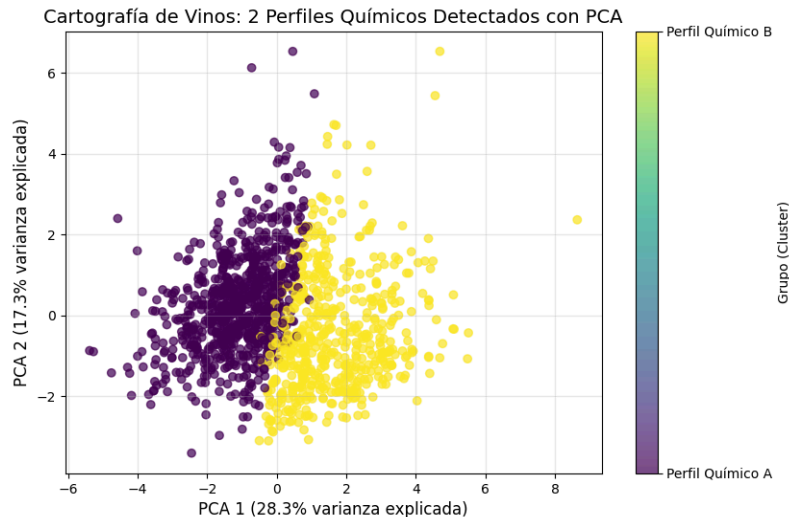
Así que hice un "re-framing" (replanteé el problema comercialmente) y decidí buscar solo 2 grandes grupos (K=2): uno "Premium" y uno "Económico". Al aplicar el algoritmo K-Means y medirlo con el Puntaje de Silueta (Silhouette Score), me dio 0.20. Es un puntaje bajito, pero tiene todo el sentido del mundo porque, como vimos al principio, casi todos los vinos son súper parecidos entre sí (notas 5 y 6).



El Mapa Final con PCA (Reduciendo las dimensiones)

Me topé con un problema práctico: como los vinos tienen 11 químicos distintos, tendría que hacer un gráfico de 11 dimensiones para ver los grupos.

Para solucionar esto, usé PCA y comprimí esas variables en solo 2 componentes. El PCA 1 explicó un 28.3% de los datos y el PCA 2 un 17.3%. Es decir, me quedé con un **45.6%** de la información original (sacrificando el 54.4% restante). Asumí esta pérdida de información de forma súper consciente, porque mi prioridad era poder dibujar un mapa 2D claro y ver visualmente si los dos perfiles de vino se separaban bien.



Mi gran descubrimiento: El Vino Premium vs Económico

Después de que la máquina armó los 2 grupos sola, analicé sus promedios y la diferencia calza perfecto con la teoría de los vinos.

El algoritmo logró aislar al Grupo 1 como el perfil "Premium". Este grupo registró la nota de calidad más alta (5.86 vs 5.46), un poquito más de alcohol (10.59% vs 10.32%) y, lo que es más notorio, mucha menos acidez volátil (0.42 vs 0.61), que es justamente lo que le da ese sabor feo a vinagre a los vinos malos. La máquina descubrió a los mejores vinos sin que yo le dijera las notas.

```

--- PERFILES QUÍMICOS Y COMERCIALES DE LAS FAMILIAS DE VINO ---
      quality    alcohol  volatile acidity
Grupo_KMeans
1      5.858974    10.594200         0.415778
0      5.464945    10.323596         0.605836

```

Mi conclusión final de esta fase

Siendo súper analítica con mis resultados, me di cuenta de que este modelo hace el mejor esfuerzo matemático posible, pero está muy limitado porque la tabla de datos original casi no tiene vinos excelentes (nota 8) o derechamente malos (notas 3 y 4). Toda la muestra es demasiado homogénea y promedio.

Para que este sistema de clustering funcione perfecto en el mundo real y defina fronteras comerciales clarísimas, sería indispensable conseguir un dataset mucho más equilibrado, que tenga muchas más muestras de vinos de alta gama y de consumo económico.

9. Conclusiones y Lecciones Aprendidas

Hacer este proyecto sola desde cero me enseñó muchísimo sobre cómo es el trabajo real en la Ciencia de Datos. Me di cuenta de que programar y aplicar los algoritmos es casi la parte "fácil", pero lo que de verdad toma tiempo, análisis y cabeza es preparar bien los datos y entender qué te están diciendo los resultados.

Aquí resumo mis mayores lecciones aprendidas durante la evaluación:

1. Limpiar datos no es borrar por borrar: Al principio, cuando vi los valores atípicos (outliers) gigantes en el azúcar y los cloruros, mi primer instinto fue borrarlos. Pero investigar el contexto del negocio me enseñó que en el mundo del vino esos valores extremos son reales y valiosos. Aprendí a usar técnicas de escalamiento en vez de simplemente eliminar filas a ciegas.
2. Las métricas pueden mentir: El desbalanceo gigante de las notas (casi todo era 5 y 6) fue mi mayor dolor de cabeza. Aprendí que si solo miraba la "Exactitud" (Accuracy), me iba a creer el cuento de que mi modelo era súper bueno. Cambiar mi evaluación a la métrica del F1-Score me abrió los ojos y me mostró la realidad: al modelo le costaba muchísimo adivinar los vinos muy raros.
3. El poder de una buena configuración: Usar GridSearchCV me demostró que los modelos por defecto casi nunca son la mejor opción. Ajustar la cantidad de árboles para evitar el subajuste y obligar al modelo a prestarle atención a las clases minoritarias con el parámetro `class_weight='balanced'`, fue clave para lograr un Random Forest mucho más honesto y robusto.
4. La magia de la Inteligencia Artificial "a ciegas": La fase de Aprendizaje No Supervisado fue la que más me sorprendió. Reducir los datos con PCA para poder graficarlos en 2D y ver cómo el algoritmo K-Means logró descubrir por sí solo el perfil exacto del "Vino Premium" (con más alcohol y menos acidez), me demostró el valor gigante que tiene esta tecnología para encontrar patrones ocultos en las empresas.

Dificultades y Recomendaciones a futuro

Mi mayor dificultad a lo largo de todas las fases fue, sin duda, lidiar con la falta de datos en los extremos (casi no habían vinos con notas 3, 4 y 8). Al final, a la máquina le es imposible aprender a reconocer algo que casi nunca ha visto.

Como recomendación a futuro, si tuviera que llevar este proyecto a una empresa vitivinícola de verdad, propondría dos grandes soluciones:

- **Conseguir más datos:** Hacer un esfuerzo para alimentar la base de datos con muchísimas más muestras de vinos excelentes y vinos deficientes.
- **Aplicar el "Re-framing":** Como expliqué en las fases anteriores, dejaría de intentar predecir 6 notas distintas y cambiaría el enfoque del negocio para clasificar los vinos en solo 3 grandes categorías: Malo, Regular y Excelente. Esto agruparía mejor los datos y haría que cualquier modelo predictivo funcionara muchísimo mejor.